



Graph Coloring with CUDA

Final Report

Names	BN	Sec
Abdelaziz Salah Mohammed	2	2
Ahmed Hosny Abdelrazik	2	1

Delivered to:

Dr/ Dina Tantawy

Eng/ Muhammed Abdullah

Date: 18/5/2024

Problem Definition:

We have a graph consisting of V vertices and E edges. Our goal is to color this graph, but we have two main constraints:

1. **Adjacent Vertices Constrains:**
    a. No two adjacent nodes should have the same color.
2. **Color Limit Constrain:**
    a. We need to use the minimum number of nodes.

and it's a fundamental problem in computer science and combinatorial optimization. It's particularly challenging because finding an optimal solution (using the fewest possible colors) is NP-hard for general graphs.

Practical Applications:

- **Register Allocation:** In compilers, graph coloring is used for register allocation where each node represents a variable, and an edge indicates a conflict (two variables cannot be stored in the same register).
- **Scheduling Problems:** In scheduling problems, tasks that cannot be performed simultaneously are represented as adjacent vertices.
- **Frequency Assignment:** In wireless networking, different frequencies must be assigned to adjacent transmitters to avoid interference.

Conquering the problem:

1. We Solved this problem in two ways:
    a. Sequential method using backtracking approach.
    b. Parallel method using **IPGC.**

Date: 18/5/2024

1. **Sequential Approach:**

- We developed an algorithm to solve this using backtracking, so our final algorithm goes as follows:
    - Read The Graph in Adjacency Matrix Representation
    - Read the number of colors need to use.
    - Create a visited array to mark the previously visited nodes.
    - Create a colors array to mark the color of each node.
    - Initialize all the colors with noColor, and visited as not visited.
    - Call **Graph Coloring => bool**
    - If we can not color the graph using the given number of color
        - Just print no solution
        - Free the memory
        - Exit
    - Else
        - Print the solution colors.
        - Free the memory
        - Print the time needed to compute this.

- **GraphColouring (graph, colours, visited, numberOfNodes, numberOfColours, numberOfColouredNodes, currentNode)**
    - This is a recursive function, which given certain node, try to assign a color to this node, by iterating over all its neighbors, If they do not have this color, then assign it, and go to the next node.
    - It returns bool:
        - True if we can color the graph.
        - False otherwise.
    - Logic:
        - numberOfColouredNodes reached the total number of nodes, then we have successfully colored all the nodes
            - return true.

Date: 18/5/2024

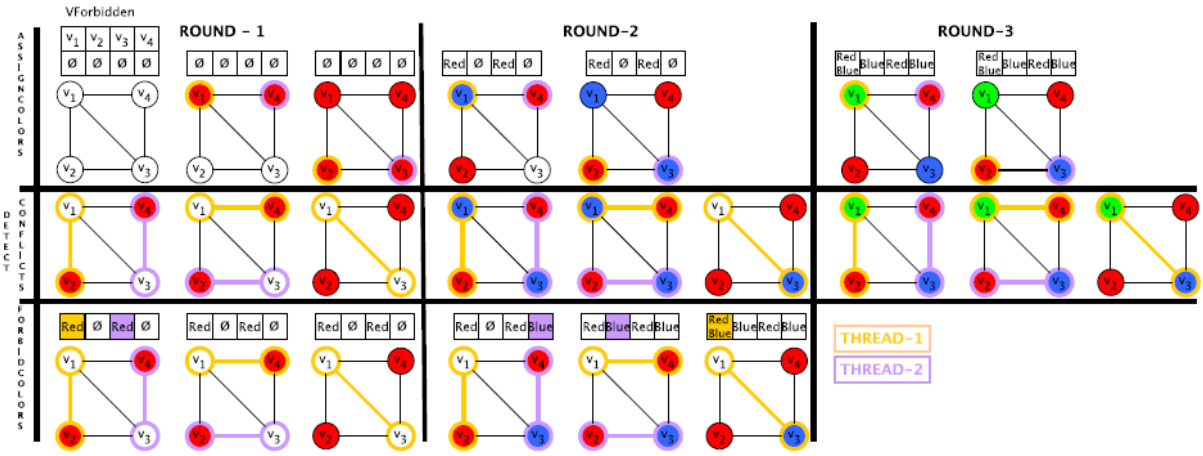
        - If the current Node is previously colored or visited before, just skip.
            - Return false.
        - Mark this node as visited.
        - Iterate over all possible colors.
            - Check if the neighbors of the current node have this color:
                - Try another color.
            - Else, then this color is safe:
                - Mark the color of the current node with this color
                - Send the next node to be colored, if we could complete the graph using this color:
                    - Return result = true
                - Else
                    - apply backtracking by removing the color of the current node.
                    - And try another color.
        - If we could not color the graph
        - Mark the current node as unvisited, and just return false.
- **Complexity of this approach:**
    - O(C^{v+1}), where C is the number of colors, and v is the number of nodes.

2. **Parallel Approach:**
    - We have looked for a paper which discusses the approach to conquer this problem, and we have found this Parallel Graph Coloring discussing an approach which can speed up the computation up to 86X
    - The theoretical complexity for our approach is the sequential complexity divided by the total number of threads * the total number of iterations needed to converge.

Date: 18/5/2024

- So, we can define it as follows:
  - $\frac{C^V \times V}{\#\,Block * \#\,Threads}$
- If we do not have C, we assume that C = V + 1 max.

- Solution Visualization:



-
- Assuming that we have 2 threads, and we have 4 vertices.
- We work with the simple approach which is Vertex-Centric, so each thread will be responsible for one vertex.
- Then we will call the Assign color kernel first, to try to color each vertex.
- Then after coloring all nodes, we will call the Detect Conflicts kernel, so if two adjacent nodes have the same color, we will remove color of one of them, and leave the other node.
- And mark that color as forbidden for this node and keep iterating till finishing the code.
- **<u>Performance Analysis:</u>**

Number of nodes	CPU Time	Our Kernel Time	OpenMP Time	Speed up
10	10ms	5ms	3.69ms	2x
100	33ms	5.73ms	4.53ms	6.6x
200	60ms	5.86ms	4.28ms	12x

<u>Date: 18/5/2024</u>

500	145ms	5.49ms	4.75ms	26.4x
1000	359ms	6.67ms	8.98ms	53x
5000	1397ms	10.49ms	25.18ms	140x
10000	2914ms	12.58ms	38.29ms	231.6x

- As the number of the nodes increases, we see huge decay in the speedup compared to the CPU time, as the number of nodes exceeds the 400, it almost does not finish on the CPU, due to the high complexity.
- One thing to consider here, is that the performance always depends on the shape of the graph, and the edges, and how complex is the graph.
- Our speedup is less than the theoretical one, and this is mainly because the data structure we have used here is the CSR, which does not support the colleasing, and it may have control divergence too, so approaches to use is to use other data structures rather than CSR like the ELL for example, so it can enhance the memory colleasing.

Date: 18/5/2024